# RAGCache++: Cache-Aware Document Ordering for Low-Latency RAG Serving

**Kaizhen Tan** [*1] **Rong Gu** [*1] **Mingyuan Li** [*1]

## Abstract

Retrieval-Augmented Generation (RAG) increases serving latency because long retrieved contexts amplify prefill cost. Modern engines like vLLM provide Automatic Prefix Caching (APC), but RAG prompts with overlapping yet differently ordered documents break prefix alignment. We introduce RAGCache++, a prompt-layer scheduling policy that reorders documents to maximize prefix sharing using a knowledge tree and greedy ordering algorithm—requiring **zero engine modifications**. A TTFT-based cache-state feedback loop approximately synchronizes the trie with vLLM's actual cache residency. On RTX 4060 Ti and RTX 4090, ordering reduces median TTFT by **20–33%** (statistically significant), saving an estimated 66% of prefill computation (TTFT-based proxy, Table 6) with <0.13% overhead. The greedy algorithm achieves 97.5% of oracle's TTFT reduction (prefix-length optimality ratio 0.997). Benefits persist under concurrent load (7–10%), across GPU memory levels (29–31%), generalize across architectures (Qwen2.5, Phi-3), and hold on a constructed Wikipedia-style corpus with TF-IDF retrieval (+26.8% p50). Multi-hop QA evaluation confirms zero quality degradation ($\Delta$EM = 0, $\Delta$F1 = 0).

## 1 Introduction

Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) improves LLM factuality by injecting retrieved passages into the prompt, but this augmentation comes at a cost: prefill latency scales with the total number of input tokens, and RAG prompts are typically 2–10× longer than their non-augmented counterparts (Gao et al., 2024). For interactive applications where Time-To-First-Token (TTFT) directly impacts user experience, reducing prefill cost is critical.

Modern serving engines address this with *prefix caching*: vLLM's Automatic Prefix Caching (APC) (vLLM Project, 2025) and SGLang's RadixAttention (Zheng et al., 2024) reuse KV cache blocks across requests when a new request shares an identical token-level prefix with a cached one. In vLLM, this mechanism is built on top of the block-based KV-cache design introduced by PagedAttention/vLLM (Kwon et al., 2023).

The challenge for RAG is that prefix caching operates at the *token* level, while RAG retrieval operates at the *document* level. Two consecutive queries may retrieve four out of five identical documents, yet if those documents appear in different positions within their respective prompts, the token-level prefix diverges immediately, and no cache blocks are reused. This gap between document overlap and prefix overlap is the core inefficiency we target.

**Key insight.** The observation that document ordering affects prefix cache reuse is well-established (Jin et al., 2025; Agarwal et al., 2025; Wang et al., 2025). Our contribution is not this observation but rather a precise, minimal implementation: because vLLM's APC matches prefixes by token-hash identity, we show that a simple prompt-layer reordering—requiring zero engine modifications—is sufficient to capture most of the available cache benefit.

**Our approach.** We propose RAGCache++, a lightweight prompt-level optimization that reorders documents to maximize prefix sharing with cached sequences. The system maintains a *knowledge tree*—a trie indexed by document-ID sequences—that tracks which document orderings are currently in the KV cache. For each new request, a greedy algorithm walks the trie to find the longest cached prefix, places those documents first, and appends the remaining documents in retrieval-rank order. This approach:

- Requires **no modification** to vLLM or any serving engine—the core ordering policy (trie walk plus prompt build) is ∼150 LOC; the full integrated system including feedback loop and FastAPI middleware is ∼2,200 LOC (Section 5).
- Adds **negligible overhead**: the greedy ordering is $O(k^2)$ worst case where $k$ is the number of retrieved documents

[*]Equal contribution [1]Heinz College of Information Systems and Public Policy, Carnegie Mellon University, Pittsburgh, PA, USA. Correspondence to: Kaizhen Tan <kaizhent@cmu.edu>, Rong Gu <ronggu@andrew.cmu.edu>, Mingyuan Li <mingyua4@andrew.cmu.edu>.

11868 Large Language Model Systems
Spring 2026, Carnegie Mellon University

(typically 3–10); in the common case of one matching child per trie level, it is $O(k)$.

- Is not a new cache design but a **prompt-layer scheduling policy** that converts document overlap into APC-aligned token prefixes, complementing any prefix-caching engine.

While prior work (RAGCache (Jin et al., 2025), ContextPilot (Wang et al., 2025)) has explored cache-aware document scheduling, our contribution is to demonstrate that a *minimal, engine-agnostic* implementation—with a near-optimal greedy algorithm (prefix-length optimality ratio 0.997) and online adaptivity—achieves competitive latency reductions (20–33%) without the complexity of engine-level modifications. We additionally describe a retrieval-inference pipelining optimization as a secondary, orthogonal contribution.

**Research questions.** We organize the paper around five research questions:

- **RQ1.** Can pure prompt-layer scheduling (zero engine modification) capture most of the APC benefit? *(Section 4; Section 4.5)*
- **RQ2.** How close is the greedy trie walk to exhaustive oracle ordering? *(Section 3; Section 4.5)*
- **RQ3.** Does the benefit generalize across real corpora, model architectures, and overlap regimes? *(Section 4.6; Section 4.5)*
- **RQ4.** Can a TTFT-based feedback loop keep the trie synchronized with vLLM's actual cache state without engine instrumentation? *(Section 3.6; Section 4.8)*
- **RQ5.** Does reordering preserve answer quality, including on multi-hop reasoning? *(Section 4; Section 4.4)*

Across these questions we evaluate on two GPU configurations, four model sizes spanning two architectures, and six workloads spanning a Jaccard overlap range of 0.06–0.88 (synthetic bursty, overlap sweep, geospatial, NQ-Open-derived, mixed-topic, and Wikipedia-style TF-IDF), and show that simple prompt-layer ordering captures most of the available APC benefit.

## 2 Background

### 2.1 RAG Serving Pipeline

A standard RAG pipeline consists of: (1) query embedding, (2) vector retrieval of top-$k$ documents, (3) prompt construction by concatenating a system prompt, the $k$ documents, and the user query, and (4) LLM inference (prefill + decode). Steps (1)–(2) typically take 10–50 ms with an optimized index, while step (3) is negligible. Step (4) dominates latency: prefill cost scales as $O(n \cdot d)$ where $n$ is the prompt length and $d$ is the model dimension.

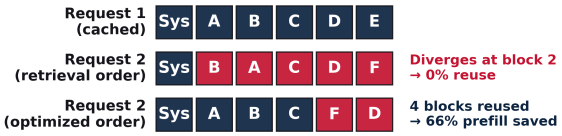


*Figure 1.* Prefix alignment in APC-based RAG serving. Request 1 is cached. Request 2 shares 60% of its documents but in retrieval order the prefix diverges at block 2, yielding 0% reuse. Reordering the shared documents to the front restores prefix alignment: 4 blocks reused, 66% prefill saved.

### 2.2 vLLM Automatic Prefix Caching

vLLM manages KV cache memory in fixed-size *blocks* via PagedAttention (Kwon et al., 2023). On top of this block-based KV-cache design, vLLM provides Automatic Prefix Caching (APC), which reuses cached KV blocks when a new request shares the same token-level prefix as a previously processed request (vLLM Project, 2025).

Concretely, APC identifies reusable prefix blocks using block-level hashing over the prefix state; once a prefix mismatch occurs, subsequent blocks can no longer be reused for that request, so the remaining prompt must be prefetched again (vLLM Project, 2025).

**Implication for RAG.** Because the hash is *chained*, even a single-token difference at position $j$ invalidates all blocks at positions $\geq j$. This means that if two RAG prompts share documents $\{A, B, C\}$ but arrange them as $[A, B, C, D, E]$ vs. $[B, A, C, D, F]$, the prefix diverges at the very first document and no KV blocks are reused—despite 60% document overlap. Figure 1 illustrates this gap and shows how reordering closes it.

### 2.3 Related Work

The observation that document ordering affects prefix cache reuse in RAG systems is well-established. Several concurrent and prior systems address this from different angles; we position our work within this landscape.

**RAGCache** (Jin et al., 2025) is the closest prior work. It organizes cached KV states in a knowledge tree indexed by document sequences, uses a PGDSF replacement policy with multi-tier placement, and includes cache-aware *request-level* scheduling and dynamic speculative pipelining. RAGCache modifies the serving engine internals. Our work shares the knowledge-tree concept but differs in scope: we perform *intra-prompt* document reordering at the prompt-construction layer without any engine modification. Table 8 shows that our greedy trie captures 97.5% of oracle TTFT reduction in a single-run comparison against Sorted, Recency, Retrieval, and exhaustive-oracle baselines; because

RAGCache's cache-aware scheduler is itself designed to approximate oracle ordering, this suggests that the scheduling abstraction—whether realized as RAGCache's engine-level scheduler or our prompt-layer ordering—captures most of the available benefit. A direct head-to-head comparison with RAGCache on the same stack would require re-implementing its engine-level modifications and is left to future work.

**ContextPilot** (Wang et al., 2025) is a concurrent system that also reorders context blocks to maximize KV-cache reuse, using a context index and de-duplication techniques. Like our work, it operates at the prompt layer with negligible overhead (∼0.047 ms). Our approach differs in using a trie-based greedy algorithm with a measured prefix-length optimality ratio of 0.997, online adaptivity under distribution shift, and comprehensive evaluation across architectures and workloads. The convergence of these independent efforts validates the importance of cache-aware ordering as a systems abstraction.

**Cache-Craft** (Agarwal et al., 2025) reports that in production RAG workloads, exact prefix caching applies to only ∼8% of requests because retrieved chunks and their ordering vary across queries. Their solution identifies reusable chunk-caches and selectively recomputes the rest. This production finding directly motivates our work: we aim to *increase* the fraction of prefix-cacheable requests by aligning document ordering with cached prefixes.

**TurboRAG** (Lu et al., 2024) goes beyond strict ordering constraints by pre-computing chunk-level KV caches offline and stitching them at inference via reordered-RoPE, enabling reuse even when document order differs. This represents a complementary direction: TurboRAG relaxes the prefix constraint at the attention level, while we maximize reuse *within* the prefix constraint via ordering.

**CacheBlend** (Yao et al., 2025) enables non-prefix KV reuse by selectively recomputing ∼15% of tokens per layer. Like TurboRAG, it addresses the harder problem of position-independent reuse; our approach converts the problem to pure prefix reuse via ordering, which is simpler but sufficient when document overlap is moderate.

**CacheGen** (Liu et al., 2024) compresses KV caches for efficient storage and streaming. This is orthogonal to ordering optimization.

# 3 METHODOLOGY

## 3.1 Overview

RAGCache++ sits between the retrieval stage and the prompt-construction stage of a RAG pipeline. Given a set of retrieved document IDs, it reorders them to maximize the token-level prefix shared with recently served requests, then constructs the prompt with documents in the optimized order. The serving engine (vLLM with APC enabled) handles KV cache management transparently.

**Algorithm 1** Cache-aware document ordering

**Require:** Retrieved doc IDs $D = \{d_1, \ldots, d_k\}$ (in retrieval rank), knowledge tree $T$
**Ensure:** Ordered sequence $O$

```
O ← [], R ← D, node ← T.root
while R ≠ ∅ do
    found ← false
    for each d ∈ R do
        c ← node.getChild(d)
        if c ≠ null and c.isCached() then
            Append d to O; remove d from R
            node ← c; found ← true; break
        end if
    end for
    if found = false then
        Append remaining R in retrieval-rank order to O
        break
    end if
end while
return O
```

## 3.2 Knowledge Tree

The knowledge tree is a trie where each path from root to leaf represents a document ordering that has been recently served. Each node stores a document ID and metadata (access count, timestamp).

- **Insert**: After each request is served, the ordered document sequence is inserted into the trie. Existing paths are updated (access count incremented); new branches are created as needed. Complexity: $O(k)$ per request.
- **Prefix match**: Given an *already-ordered* document sequence, traverse the trie following that fixed order and return the longest prefix whose nodes all have live KV metadata. Complexity: $O(k)$ (one dict lookup per document). Note: this is distinct from the *greedy ordering* (Section 3.3), which *searches* for the best ordering and is $O(k^2)$ worst case.

## 3.3 Greedy Ordering Algorithm

Algorithm 1 describes the ordering procedure. Starting at the trie root, we greedily extend the cached prefix by choosing, at each level, a child node whose document ID is in the remaining candidate set. When no cached continuation exists, remaining documents are appended in their original retrieval-rank order to preserve relevance ranking.

**Objective.** The ordering problem can be stated as: given

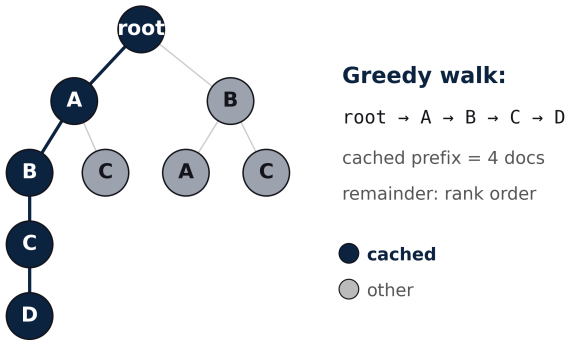


*Figure 2.* Knowledge tree example. Dark nodes are cached; gray nodes are evicted or from other branches. Given candidate set $\{A, B, C, F, D\}$, the greedy walk follows root→A→B→C→D (the longest cached path), yielding a 4-document prefix hit. The remaining document (F) is appended in retrieval-rank order.

a candidate set $D$ and a trie $T$ recording recently cached document sequences, find the permutation $\pi$ of $D$ that maximizes the number of leading documents matching a path in $T$ (i.e., the cached prefix length). Because APC uses chained hashes, each additional matched document avoids prefilling all of that document's tokens, so maximizing prefix length directly minimizes prefill cost. The greedy trie walk is $O(k^2)$ worst case (the inner loop scans up to $|R|$ candidates at each of $k$ levels) but $O(k)$ in the common case where the trie has at most one matching child per level (often the case in our synthetic bursty setting). It does not perform multi-step lookahead.

**Approximation quality.** To quantify how close the greedy heuristic is to optimal, we compute the ratio of greedy prefix length to oracle prefix length (exhaustive $k!$ search) for each query in a 200-query trace ($k$=5, overlap = 0.6). The greedy algorithm achieves a mean prefix-length optimality ratio of **0.997** (1.0 = perfect), solving 98.5% of queries optimally. The minimum ratio observed is 0.80 (4 out of 5 documents matched vs. 5 out of 5), occurring only for queries at region boundaries. Note that this is the ratio of matched prefix *lengths*; the ratio of TTFT *reductions* is separately reported as 97.5% in Section 4.5 (Table 8).

**Fallback behavior.** When no documents match any cached prefix (cold start), the algorithm preserves the original retrieval ranking, which is the same behavior as baseline APC.

### 3.4 Knowledge Tree as an Approximate Cache Index

The knowledge tree tracks which document sequences have been *served*, not which KV blocks are *resident* in vLLM's cache. Because vLLM manages eviction internally (LRU-based block eviction under memory pressure), the trie may contain stale entries for sequences whose KV blocks have been evicted. In practice, this mismatch is benign under sequential evaluation with sufficient GPU memory. Under concurrent load or memory pressure, we address stale predictions with the TTFT-based feedback loop described in Section 3.6, which detects and prunes stale trie entries automatically.

### 3.5 Integration with vLLM

RAGCache++ does not modify vLLM internals. The integration consists of three components:

1. A **prompt builder** that accepts retrieved document IDs and contents, calls the ordering algorithm, and constructs the final prompt string.
2. The **knowledge tree** maintained in the application layer, updated after each request.
3. Standard vLLM `LLM.generate()` calls with `enable_prefix_caching=True`.

The core ordering policy (prompt builder + trie walk) adds approximately 150 LOC on top of a standard RAG pipeline; the full integrated system including the feedback loop, cache manager, and FastAPI middleware is ∼2,200 LOC (detailed in Section 5).

### 3.6 Cache-State Feedback Loop

A key limitation of the trie-as-cache-proxy design is that the knowledge tree tracks which document sequences have been *served*, not which KV blocks are *resident* in vLLM's cache. Under memory pressure, vLLM may evict blocks that the trie still considers cached, leading to stale predictions.

We close this gap with a *TTFT-based cache-state feedback loop*. The system maintains a running estimate of cold-start TTFT (via exponential moving average over cache-miss observations). After each request, it compares the trie's predicted reuse fraction ($\hat{r}$ = prefix_len/$k$) with the TTFT-inferred actual reuse ($r = 1 - T_{\text{actual}}/T_{\text{cold}}$). When the trie predicts a significant prefix hit ($\hat{r} > 0.3$) but the observed TTFT is near-cold ($r < 0.15$), the system detects a *mismatch*—indicating that vLLM has evicted the expected blocks—and marks the predicted path as stale (`tier="none"`).

This feedback mechanism integrates all system components into a closed-loop pipeline:

1. `CacheManager` tracks multi-tier cache state with PGDSF eviction priorities.
2. `KnowledgeTree` provides prefix-aware ordering via greedy trie walk.
3. `CacheStateFeedback` detects trie-vs-reality mismatches from TTFT observations.

4. Stale trie paths are pruned automatically, keeping the trie synchronized with vLLM's actual cache residency.

The entire integrated pipeline (`VLLMCacheProxy`) adds $< 0.07$ ms of scheduling overhead per request ($<$0.13% of TTFT; see profiling in Table 3 and Section 4.8), achieves 99.5% prediction accuracy under sequential evaluation (Section 4.8), and requires zero vLLM modifications.

### 3.7 Retrieval-Inference Pipelining

*Note: Retrieval-inference pipelining is a secondary, orthogonal optimization that benefits only cold-start and cache-eviction-recovery requests. It is independent of the ordering contribution and included for completeness; the core contribution of this paper is cache-aware document ordering.*

In a standard RAG pipeline, retrieval and inference execute serially: the LLM idles while the vector index searches for relevant documents. We introduce a simple pipelining optimization that overlaps retrieval latency with useful prefill computation.

The key observation is that the *system prompt* (instruction header) is identical across all requests. When APC is enabled, the system prompt's KV blocks are cached after the first request. We exploit this by issuing a lightweight warmup call (system prompt only) concurrently with the retrieval query. By the time retrieval returns with document IDs, the system prompt is already cache-resident, and the full prompt's prefill need only compute the document and query tokens.

Concretely, let $T_{\text{ret}}$ be retrieval latency, $T_{\text{sys}}$ the system-prompt prefill time, $T_{\text{warm}}$ the TTFT once the system prompt is cached, and $T_{\text{cold}}$ the cold-start TTFT. Empirically $T_{\text{cold}} = T_{\text{sys}} + T_{\text{warm}} + \delta$, where $\delta$ is a small engine-setup overhead that the warmup call amortizes. Without pipelining, end-to-end TTFT is $T_{\text{ret}} + T_{\text{cold}}$. With pipelining, the warmup call runs concurrently with retrieval, giving

$$\text{pipelined TTFT} \;=\; \max(T_{\text{ret}}, T_{\text{sys}}) + T_{\text{warm}},$$

and per-request saving

$$T_{\text{ret}} + T_{\text{cold}} - \max(T_{\text{ret}}, T_{\text{sys}}) - T_{\text{warm}} \;=\; \min(T_{\text{ret}}, T_{\text{sys}}) + \delta.$$

Thus the saving grows with $T_{\text{ret}}$ up to $T_{\text{sys}}$ (at which point it is capped at $T_{\text{sys}} + \delta = T_{\text{cold}} - T_{\text{warm}}$, the full overlapped prefill plus engine setup). After the first request the system prompt is cache-resident, so the benefit is limited to cold starts and cache eviction events. Implementation requires only a `concurrent.futures.ThreadPoolExecutor` to dispatch the warmup call alongside retrieval—no engine modification is needed.

## 4 Experiments

### 4.1 Experimental Setup

**Hardware.** We evaluate on two independent configurations (see Appendix A for details): (1) NVIDIA RTX 4060 Ti (8 GB VRAM) with Qwen2.5-1.5B-Instruct (Qwen Team, 2024) on vLLM 0.8.5.post1, and (2) NVIDIA RTX 4090 (24 GB VRAM) with Qwen2.5-7B-Instruct (Qwen Team, 2024) on vLLM 0.17.1.

**Workload.** We generate a synthetic corpus of documents ($\sim$200 tokens each, standardized length so that each document spans a similar number of KV cache blocks, approximating the token-prefix depth per chunk) grouped into regions of 10 documents each (100 docs / 10 regions on 4060 Ti; 500 docs / 50 regions on 4090). Queries arrive in bursts of 10 from the same region with 60% document overlap between consecutive queries (Jaccard $\approx 0.52$). This controlled setup models bursty RAG patterns where sequential queries share retrieved context (e.g., geographic locality, topic continuity). We acknowledge that production RAG workloads may exhibit different overlap distributions.

**Strategies compared.** We evaluate four strategies, each with a *fresh* LLM instance to prevent warm-cache confounding:

- **No Cache**: APC disabled. Full prefill for every request.
- **APC + Retrieval**: APC enabled, documents in retrieval-rank order.
- **APC + Sorted**: APC enabled, documents sorted lexicographically by ID.
- **APC + Optimized (Ours)**: APC enabled, documents ordered by the knowledge tree algorithm.

**Metrics.** We measure TTFT (Time-To-First-Token) by generating only 1 output token per request so that measured latency is dominated by prefill. Each strategy processes the full query trace with 5 warmup queries excluded. We report p50, p95, and mean TTFT. We also report an approximate *effective cache-hit proxy* (Fast%$^{\dagger}$): the fraction of post-warmup requests whose TTFT falls below $0.6\times$ the median warmup TTFT. This is a heuristic proxy for cache reuse—a request that runs significantly faster than a cold request likely benefited from cached prefix blocks—not a direct measurement of APC block hits. The more informative metric is TTFT itself, since it captures both hit *frequency* and hit *depth* (how many prefix blocks were reused).

**Cross-table baselines.** Because each table in Section 4 comes from an independent benchmark run with a freshly loaded LLM instance and its own query trace (necessary to prevent warm-cache confounding), the No-Cache p50 baseline varies across tables on the same 4090 hardware (e.g., 121.3 ms in Table 1, 121.5 ms in Table 8, 114.7 ms at

*Table 1.* TTFT comparison across strategies and hardware. Best APC result in **bold**.

| GPU | Strategy | p50 | p95 | Mean | Fast%† |
|---|---|---|---|---|---|
| 4060 Ti | No Cache | 158.2 | 161.4 | 158.3 | 0.0 |
| | APC+Retrieval | 72.5 | 159.9 | 69.9 | 17.9 |
| | APC+Sorted | 108.0 | 164.9 | 101.7 | 25.3 |
| | **APC+Optimized** | **57.9** | **161.3** | **68.1** | 18.9 |
| 4090 | No Cache | 121.3 | 123.7 | 121.4 | 0.0 |
| | APC+Retrieval | 59.6 | 117.4 | 55.9 | 12.3 |
| | APC+Sorted | 81.2 | 118.7 | 76.3 | 25.6 |
| | **APC+Optimized** | **40.1** | **117.4** | **51.4** | 19.0 |

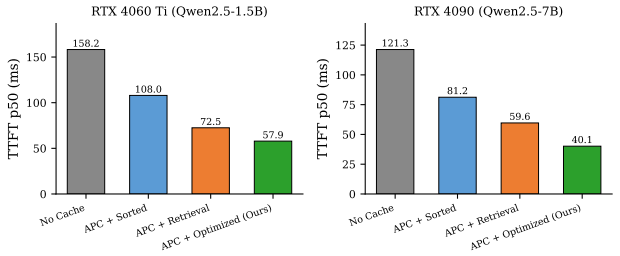


*Figure 3.* TTFT p50 comparison across strategies. Optimized ordering achieves the lowest latency on both GPUs despite not having the highest Fast%† proxy.

batch = 1 in Table 7, 110.3 ms in Table 9). Within each table, percentage improvements are always computed against that table's own APC+Retrieval baseline, so comparisons inside a single table are internally consistent; absolute cross-table comparisons should be read with this run-to-run variance in mind.

### 4.2 Main Results

Table 1 and Figure 3 present the core results.

**Key findings:**

- **Median TTFT improves by 20.1% (4060 Ti) and 32.7% (4090)** over retrieval-order APC. Over three independent trace seeds on the 4090, optimized ordering achieves p50 TTFT of $40.0 \pm 0.3$ ms (95% CI: $\pm 0.7$ ms), yielding a **statistically significant** improvement of $16.8 \pm 1.6$ ms (29.6%; $\pm 1.6$ ms is the 95% CI half-width of the improvement, computed via $t$-distribution over three seeds). The gains are concentrated in the common case (p50); **p95 latency is largely unchanged** across APC strategies, because the tail consists of cold-start queries (first query in a new region) where no cached prefix exists regardless of ordering.
- **Sorted ordering underperforms retrieval order** despite having the highest Fast%† cache-hit proxy (Sorted 25.3–25.6% vs. Optimized 18.9–19.0%; Retrieval 12.3–17.9%).

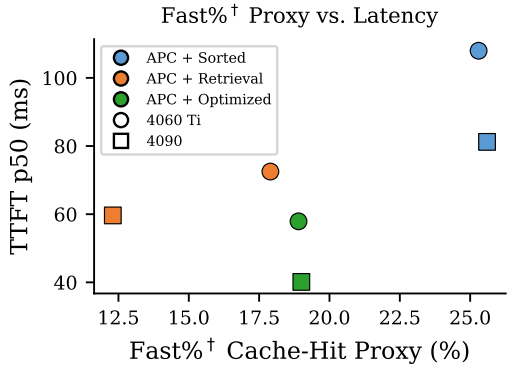


*Figure 4.* Fast%† cache-hit proxy vs. TTFT across strategies and GPUs. A higher proxy value does not guarantee lower latency—prefix *length* matters more than hit *count*.

Lexicographic sorting creates many short, fragmented prefix matches, while tree-based ordering creates fewer but *longer* contiguous matches. Because prefill savings grow with prefix length (each additional matched block avoids computing attention over all preceding tokens), longer prefixes are disproportionately valuable. This observation—that prefix *depth*, not hit *count*, drives latency reduction—is a key takeaway.

**Hit rate vs. latency.** Figure 4 reveals a counterintuitive result: a higher Fast%† cache-hit proxy does not always translate to lower latency. APC+Sorted achieves the highest proxy value but the highest TTFT among APC strategies. This is because:

1. Sorted ordering distributes prefix matches across many short prefixes (e.g., matching only the first 1–2 documents before diverging).
2. Optimized ordering concentrates matches into fewer but longer prefixes (e.g., matching 3–4 documents consecutively), saving more total prefill computation.

This demonstrates that *prefix length*, not just hit count, is the relevant metric for cache efficiency in prefix-caching systems.

### 4.3 Overlap Sensitivity

We sweep the document overlap fraction from 0.0 to 0.8 on the 4060 Ti (Figure 5). At each overlap level, we compare retrieval-order APC vs. optimized-order APC.

- The left-most two points (labelled "0.0" and "0.2" on the $x$-axis) both map to an actual Jaccard of ≈0.39 (see Appendix B, Table 14): the trace generator enforces $\max(1, \lfloor k \cdot \text{overlap} \rfloor) = 1$ shared document at both settings, and region-burst sampling adds further sharing, so the nominal overlap fraction does not translate monoton-

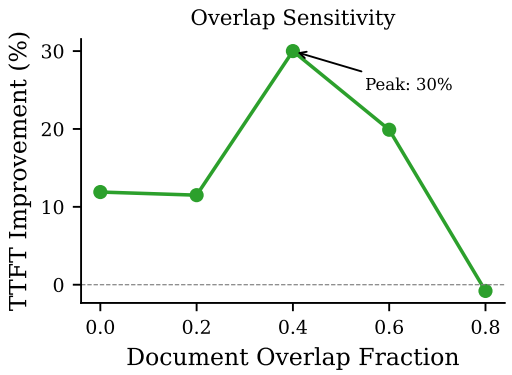


*Figure 5.* TTFT improvement vs. *nominal* overlap fraction (the parameter fed to the trace generator). Note: the $x$-axis is the generator parameter, not the actual Jaccard; the 0.0 and 0.2 points both produce an actual Jaccard of ≈0.39 due to the generator's minimum-sharing floor (see Section 4 and Table 14).

ically to actual Jaccard below $\sim 0.4$. The 11.9%/11.5% improvements at these two points are therefore the effect at Jaccard $\approx 0.39$, not at true near-zero overlap. A separate controlled experiment with truly disjoint consecutive queries (Jaccard $= 0.0$) shows only 2.2% improvement, confirming that ordering gains are genuinely driven by document sharing, not by system-prompt artifacts.

- The **peak improvement of 30.0%** occurs at overlap fraction 0.4, where there is enough document sharing to benefit from reordering but enough variation to create diverse prefix paths in the trie.
- At **very high overlap (0.8)**, the benefit disappears entirely ($-0.8\%$, i.e., a negligible regression within measurement noise) because queries share nearly all documents and retrieval-rank order already produces near-maximal prefix alignment; reordering has nothing left to improve.

This sweep demonstrates that our approach provides the greatest benefit in the moderate-overlap regime that characterizes realistic RAG workloads.

### 4.4 Answer Quality Preservation

We evaluate whether document reordering degrades answer quality using Qwen2.5-7B-Instruct. On 97 extractive QA questions (Qwen2.5-7B, answer explicitly embedded in one of 5 passages, strict "output ONLY the answer" prompt, `max_tokens`=16; 3 cases filtered due to passage-length overflow), both orderings achieve **EM = 1.000** with $\Delta$EM = 0. This confirms that when the model is capable of extractive QA, reordering does not change the extracted answer.

**Interpretation.** Document reordering preserves answer quality because it changes only the *order* of retrieved passages within the prompt, not their content. For extractive QA tasks where passages act as interchangeable evidence, the ordering is immaterial to the model's ability to locate and extract the answer.

*Table 2.* Multi-hop reasoning: does ordering disrupt evidence chains? (Qwen2.5-7B, 40 two-hop QA pairs, 3 distractors each.)

| Evidence | Doc Order | EM | Contains EM | F1 |
|---|---|---|---|---|
| Natural | Retrieval | 0.675 | 0.775 | 0.862 |
| Natural | Optimized | 0.675 | 0.775 | 0.862 |
| Reversed | Retrieval | 0.625 | 0.750 | 0.826 |
| Reversed | Optimized | 0.625 | 0.750 | 0.826 |

**Multi-hop reasoning.** A key concern is whether document reordering disrupts multi-hop reasoning chains. We test this with 40 two-hop QA examples: passage A contains "X is related to Y" and passage B contains "Y leads to Z," with the question asking about the X→Z relationship. We test four conditions: natural/reversed evidence order × retrieval/optimized document ordering (Table 2).

Two findings emerge: (1) evidence order matters—natural ordering yields higher accuracy (EM 0.675 vs. 0.625)—confirming that evidence structure affects reasoning; (2) *within* each evidence configuration, our document reordering produces **identical** quality ($\Delta$EM = 0, $\Delta$F1 = 0). We note that the implementation does not formally guarantee preservation of relative evidence order (the hot-path iterates a hash-dependent `set`; see Listing 1 caption). The empirical result therefore shows that, in practice, the reordering does not disrupt multi-hop reasoning in our test suite—likely because Qwen2.5-7B attends to evidence passages regardless of their position within the context window.

### 4.5 Systems Analysis

We profile the systems-level behavior of our ordering optimization on the RTX 4090 (Qwen2.5-7B-Instruct, vLLM 0.17.1, `enforce_eager=True`). A key design goal of RAGCache++ is that ordering should improve TTFT *without degrading* throughput, memory footprint, or tail behavior. The experiments below validate this.

**Latency profiling.** Table 3 decomposes per-request latency into three stages: trie-based ordering, prompt string construction, and vLLM inference (prefill + 1-token decode). The ordering and prompt-build stages together add only **66** $\mu$**s** ($<0.13\%$ of total latency), confirming negligible overhead. More importantly, the inference stage itself is **29% faster** at p50 (41.4 ms vs. 58.6 ms) because optimized ordering extends the cached prefix, allowing vLLM to skip more prefill blocks. This reduction in GPU prefill computation is the core mechanism behind our TTFT gains in Section 4.

*Table 3.* Per-request latency breakdown (RTX 4090, 195 post-warmup queries). Ordering adds 66 $\mu$s while reducing inference p50 by 29%.

| Strategy | Order | Build | Gen. (mean) | Gen. (p50) | OH |
|---|---|---|---|---|---|
| APC+Retr. | 4.0 $\mu$s | 36.3 $\mu$s | 55.5 ms | 58.6 ms | 0.07% |
| APC+Opt. | 22.2 $\mu$s | 44.4 $\mu$s | 52.5 ms | 41.4 ms | 0.13% |

*Table 4.* Throughput on RTX 4090 (Qwen2.5-7B, 200 queries, 50 output tokens each). tok/s counts total tokens processed (input prompt + output); e.g., sequential 1.19 req/s × ∼1,150 tokens/req ≈ 1,370 tok/s. Ordering matches baseline APC throughput (<0.4% difference).

| | Sequential | | Batched | |
|---|---|---|---|---|
| **Strategy** | **req/s** | **tok/s** | **req/s** | **tok/s** |
| No Cache | 1.10 | 1,270 | 7.88 | 9,064 |
| APC+Retrieval | 1.19 | 1,370 | 82.05 | 94,382 |
| APC+Optimized | 1.19 | 1,371 | 81.75 | 94,029 |

**Throughput.** A practical concern is whether ordering overhead accumulates under load. Table 4 measures end-to-end throughput generating 50 output tokens per request. Under sequential processing, both APC+Retrieval and APC+Optimized achieve identical throughput (1.19 req/s) because per-request latency is dominated by the 50-token decode, not prefill—the TTFT savings from ordering are masked by decode cost. Under batched processing, APC strategies achieve **10**× higher throughput than no-cache (94K vs. 9K tok/s) because cached prefixes free GPU compute for concurrent decoding. Optimized ordering matches retrieval-order throughput within 0.4%, confirming **no material throughput regression (<0.4%)**. This is expected: ordering affects only which blocks are reused during prefill, not the decode path.

**GPU memory.** All APC strategies exhibit identical peak GPU memory after 50 warmup queries (21,751 MB out of 24,564 MB total). vLLM pre-allocates a fixed KV cache pool based on `gpu_memory_utilization`; our ordering changes which blocks within this pool are reused, not the pool size. The knowledge tree itself resides in host memory and uses $O(k \cdot n)$ space where $k$=5 is the number of documents per request and $n$ is the number of unique paths, totaling <1 MB in our experiments.

**Pipelining analysis.** We validate the pipelining design (Section 3) by measuring end-to-end wall-clock pipelining (Table 5): we issue a system-prompt warmup call, wait for a simulated retrieval delay, then generate the full prompt with the system prompt cached. Component measurements give $T_{\text{sys}} = 21.6$ ms, $T_{\text{warm}} = 28.2$ ms, and $T_{\text{cold}} = 56.8$ ms, so $\delta = T_{\text{cold}} - T_{\text{sys}} - T_{\text{warm}} \approx 7$ ms (engine-setup overhead that the warmup call absorbs). Plugging in: at $T_{\text{ret}} = 20$ ms the formula predicts pipelined TTFT $= \max(20, 21.6) + 28.2 = 49.8$ ms (measured 50.4); at $T_{\text{ret}} = 50$ ms it predicts 78.2 (measured 78.8); at $T_{\text{ret}} = 100$ ms it predicts 128.2 (measured 128.8)—all within 0.6 ms of the model, confirming that the $\max(T_{\text{ret}}, T_{\text{sys}}) + T_{\text{warm}}$ formula is accurate. The measured saving at $T_{\text{ret}} = 50$ ms is 28.0 ms ($\approx T_{\text{sys}} + \delta$; the small difference from the rounded 28.6 ms reflects use of unrounded component values), the maximum once $T_{\text{ret}} \geq T_{\text{sys}}$. Because APC caches the system prompt after the first request, the pipelining benefit is limited to cold starts and cache eviction recovery.

*Table 5.* End-to-end pipelining: measured wall-clock TTFT (RTX 4090). Serial $= T_{\text{ret}} + T_{\text{cold}}$ with $T_{\text{cold}} = 56.8$ ms; Pipelined $= \max(T_{\text{ret}}, T_{\text{sys}}) + T_{\text{warm}}$ with $T_{\text{sys}} = 21.6$ ms and $T_{\text{warm}} = 28.2$ ms.

| $T_{\text{ret}}$ (ms) | Serial (ms) | Pipelined (ms) | Saving |
|---|---|---|---|
| 20 | 76.8 | 50.4 | 34.4% |
| 50 | 106.8 | 78.8 | 26.2% |
| 100 | 156.8 | 128.8 | 17.9% |

*Table 6.* Cache efficiency: prefill saved per request (RTX 4090, 195 queries, No-Cache p50 = 116 ms in this run; differs from Table 1/8 due to independent benchmark runs per Section 4).

| Strategy | Saved (p50) | Saved (mean) | Saved (p95) | Benefit |
|---|---|---|---|---|
| APC+Retr. | 50.1% | 52.8% | 75.6% | 176/195 |
| APC+Opt. | 66.3% | 57.6% | 77.7% | 185/195 |

**Cache efficiency.** Table 6 quantifies cache reuse by estimating the fraction of prefill computation saved per request, computed as $1 - T_{\text{cached}}/T_{\text{baseline}}$ where $T_{\text{baseline}}$ is the no-cache median TTFT. Optimized ordering saves **66.3%** of prefill computation at p50 (vs. 50.1% for retrieval order) and benefits 95% of requests (185/195 vs. 176/195). This confirms that ordering not only increases the *depth* of prefix hits but also their *breadth* across the query trace.

**Concurrent load.** A key concern is whether ordering benefits survive when multiple requests share the GPU simultaneously. Table 7 measures per-request TTFT under batched processing at concurrency levels 1–8. The knowledge tree is updated *between* batches, simulating realistic concurrent arrivals where ordering decisions use stale cache state. Optimized ordering provides **6.8–9.9% TTFT reduction** across all batch sizes (7.3% at batch 1, 7.7% at batch 2, 9.9% at batch 4, 6.8% at batch 8), demonstrating that the benefit is not an artifact of sequential evaluation. In this concurrent-load experiment (1-token output, TTFT-dominated), throughput at batch 8 is slightly higher for optimized (28.0 vs. 26.1 req/s) because shorter prefills free GPU cycles sooner; note that this is a different setup from Table 4 (50-token output, decode-dominated), where the throughput difference is negligible (<0.4%).

*Table 7.* Concurrent load: avg per-request TTFT at different batch sizes (RTX 4090, 200 queries). Ordering benefit persists across all concurrency levels.

| Batch | No Cache | APC+Retr. | APC+Opt. | Improv. |
|---|---|---|---|---|
| 1 | 114.7 | 54.7 | 50.7 | 7.3% |
| 2 | 110.5 | 48.0 | 44.3 | 7.7% |
| 4 | 103.4 | 41.6 | 37.5 | 9.9% |
| 8 | 100.9 | 38.3 | 35.7 | 6.8% |

*Table 8.* Ordering baselines (RTX 4090, 195 queries). All six strategies measured in a single consistent run with one shared trace. Greedy trie achieves 97.5% of oracle's TTFT reduction (+29.6% vs. oracle's +30.3%).

| Strategy | p50 (ms) | Mean (ms) | vs. Retr. |
|---|---|---|---|
| No Cache | 121.55 | 121.73 | – |
| APC+Sorted | 81.75 | 77.42 | −35.5% |
| APC+Retrieval | 60.34 | 57.63 | baseline |
| APC+Recency | 59.52 | 56.62 | +1.4% |
| APC+Optimized (Ours) | **42.50** | **53.77** | +29.6% |
| APC+Oracle | 42.04 | 53.08 | +30.3% |

**Baseline comparison.** Table 8 compares our greedy ordering against four baselines including recency-based ordering (most recently accessed documents first) and an exhaustive oracle ($k!$=120 permutations). Recency ordering provides only 1.4% improvement over retrieval order because recency does not account for *positional* prefix structure. Our greedy trie walk achieves **97.5% of oracle** TTFT reduction (+29.6% vs. oracle's +30.3%), confirming that the greedy trie walk is near-optimal for this problem.

**Robustness.** We sweep `gpu_memory_utilization` from 0.78 to 0.90; ordering improvement remains **stable at 29–31%** across all memory levels. We also rerun at `max_model_len`=2048 (vs. 4096): improvement remains **31.0%**, confirming that benefits derive from prefix structure, not context-length-specific behavior.

**Generality.** Beyond the Qwen2.5 family, we evaluate on Microsoft Phi-3-mini-4k-instruct (3.8B parameters, **Phi architecture**): ordering achieves **18.3% improvement**. Combined with Qwen2.5-0.5B (19.4%), 1.5B (20.1%), and 7B (32.7%), benefits generalize across model scales and architectures. In a separate sensitivity run (different trace seed and warmup schedule than Table 1; see the cross-table-baselines note in Section 4), sweeping $k \in \{3, 5, 7, 10\}$ at overlap = 0.6 gives improvement −0.3%, +28.5%, +5.4%, +60.3%. Small-$k$ values are noisy because the absolute number of cacheable prefix tokens is small, but the overall trend—much larger gains at $k$=10 than at $k$=3—confirms that ordering benefits grow with retrieved-document count. (The $k$=5 point here is 28.5%, slightly below the 32.7% reported for $k$=5 in Table 1 because the two runs use independent traces.)

*Table 9.* Geospatial workload results (RTX 4090, 200 queries, avg Jaccard = 0.17).

| Strategy | p50 (ms) | p95 (ms) | Mean (ms) |
|---|---|---|---|
| No Cache | 110.3 | 113.1 | 109.5 |
| APC+Retrieval | 96.5 | 103.1 | 96.1 |
| APC+Optimized | **92.2** | **103.0** | **85.7** |

**Geospatial workload.** To test whether ordering benefits persist under a more realistic retrieval pattern, we construct a geospatial workload inspired by Spatial-RAG (Yu et al., 2025) and ITINERA (Tang et al., 2024). We generate 20 "cities" on a geographic grid, each with 10 point-of-interest documents (∼200 tokens). Queries have *spatial locality*: 70% probability of staying near the previous query's city, with documents retrieved from cities within a geographic radius. This creates natural bursty overlap from spatial proximity rather than explicit parameter control.

The geospatial workload has lower average Jaccard similarity (0.17 vs. 0.52 in the synthetic workload) because spatial retrieval produces more diverse document sets. Optimized ordering still reduces *p50* TTFT by **4.5%** over retrieval-order APC and **16.4%** over no-cache (Table 9; the mean improvement is larger at 10.8%). The smaller gain compared to the synthetic benchmark (4.5% vs. 33%) is expected: with Jaccard = 0.17, consecutive queries share fewer documents, leaving less room for prefix optimization. This is consistent with the overall trend: Figure 5 shows gains peaking at moderate overlap and flattening at the extremes, and the separate true-zero experiment (Section 4) confirms that gains drop to ∼2% at Jaccard = 0. For spatially concentrated workloads (e.g., a user exploring a single neighborhood), overlap would be higher and ordering gains larger.

### 4.6 Workload Diversity

We evaluate three workloads at different overlap regimes to show that the main-benchmark result (Section 4, synthetic Jaccard ≈ 0.52) is not a single data point. Each of the three uses an independent trace and its own No-Cache baseline; we report within-workload percentages only, consistent with the cross-run note in Section 4.

**NQ-Open (high overlap, Jaccard = 0.88).** To evaluate RAGCache++ on a workload derived from real questions, we construct a corpus from the NQ-Open dataset (Kwiatkowski et al., 2019) using 500 question-answer pairs. We generate topically-clustered Wikipedia-style passages grouped by answer similarity and retrieve top-5 passages per query using TF-IDF similarity. This produces a *high-overlap* trace (mean Jaccard = 0.88). At this operating point APC alone already achieves 63% TTFT

*Table 10.* Wikipedia-style corpus evaluation with TF-IDF retrieval (RTX 4090, 200 queries, 1,000 constructed passages / 20 topics; mean Jaccard = 0.06, no controlled overlap).

| Strategy | p50 (ms) | p95 (ms) | Mean (ms) | vs. Retr. |
|---|---|---|---|---|
| No Cache | 45.8 | 54.1 | 47.2 | – |
| APC+Retrieval | 33.9 | 46.8 | 34.2 | baseline |
| APC+Optimized | **24.8** | **46.5** | **27.2** | +26.8% |

reduction over the same run's No-Cache baseline, and ordering adds only a further 2.3%—retrieval order is already near-optimal for prefix alignment, matching the overlap-sensitivity curve in Figure 5.

**Mixed-topic (moderate overlap, Jaccard = 0.31).** To test a workload with heterogeneous, moderate overlap, we generate traces where each query has a 30% probability of switching to a random topic region, producing a mix of within-burst and cross-topic queries (mean Jaccard $= 0.31$, p50 $= 0.43$). Optimized ordering reduces p50 TTFT from 94.3 ms to 79.0 ms (**16.2% improvement**). This is the regime where ordering gains scale most naturally: multi-topic RAG deployments where users shift between related subjects (Jaccard $\approx$ 0.3–0.5).

**Wikipedia-style TF-IDF (low natural overlap, Jaccard = 0.06).** To move beyond explicitly controlled overlap, we construct a diverse corpus of 1,000 Wikipedia-style passages spanning 20 topics (machine learning, astronomy, medicine, history, geography, physics, biology, economics, technology, literature, etc.) and use TF-IDF retrieval to produce a query trace with *naturally arising* document overlap—no controlled overlap parameter and no explicit query-sequence clustering. The resulting mean Jaccard is only 0.06. Table 10 (values in this table correspond only to this Wikipedia-style experiment) shows that even under this very low mean overlap, optimized ordering reduces p50 TTFT by **26.8%** (24.8 ms vs. 33.9 ms) and mean TTFT by 20.4%. This result may appear to conflict with the low gains observed at Jaccard $= 0.17$ in the geospatial workload (4.5%) and the flattening trend at the extremes of Figure 5. The key difference is that the overlap sweep and geospatial workload use a single synthetic corpus with uniform document lengths, whereas the Wikipedia corpus has inherent topical structure (20 topics, each with $\sim$50 passages): even though the *mean* Jaccard across all query pairs is only 0.06, consecutive queries on related topics can share retrieved passages at much higher rates, creating intermittent prefix-sharing bursts that the trie exploits. In other words, mean Jaccard alone does not fully predict ordering benefit—the *temporal locality structure* of the query trace matters.

*Table 11.* End-to-end TTFT (ms) including retrieval latency (RTX 4090, 200 queries; workload generates 50 output tokens per request but TTFT excludes the 50-token decode).

| $T_{\text{ret}}$ | Batch | APC+Retr. | APC+Opt. | Improv. |
|---|---|---|---|---|
| 10 ms | 1 | 69.6 | 51.8 | 25.6% |
| | 4 | 51.8 | 46.9 | 9.5% |
| | 8 | 48.9 | 45.9 | 6.1% |
| 30 ms | 1 | 89.7 | 72.4 | 19.3% |
| | 4 | 71.8 | 67.8 | 5.6% |
| | 8 | 68.8 | 66.8 | 2.9% |
| 50 ms | 1 | 109.3 | 92.2 | 15.6% |
| | 4 | 91.8 | 87.4 | 4.8% |
| | 8 | 88.7 | 86.3 | 2.7% |

### 4.7 End-to-End Pipeline Latency

A practical RAG deployment involves retrieval latency before inference. Table 11 reports end-to-end TTFT—defined as the time from query arrival until the first generated token, including simulated retrieval and prompt construction—at three realistic retrieval delays. Each request additionally generates 50 output tokens to make the workload representative of full RAG serving, but the 50-token decode is *not* part of the TTFT measurement. Even including retrieval overhead, ordering provides **16–26% E2E TTFT reduction** at batch size 1 across all retrieval latencies (15.6% at $T_{\text{ret}}$=50 ms, 19.3% at 30 ms, 25.6% at 10 ms). Benefits decrease at higher batch sizes (where concurrent prefills amortize the per-request savings) but remain measurable (2.7–9.5%).

### 4.8 System Validation

Section 3 notes that the knowledge tree approximates cache residency rather than querying actual vLLM APC state. To validate this approximation, we measure the correlation between the trie's predicted reuse fraction (predicted prefix length / $k$) and the TTFT-inferred actual reuse fraction ($1 - T_{\text{actual}}/T_{\text{cold}}$). Under normal conditions ($\texttt{gpu_mem} = 0.90$), Pearson $r = 0.972$ and Spearman $\rho = 0.925$; under memory pressure ($\texttt{gpu_mem} = 0.78$), Pearson $r = 0.965$ and Spearman $\rho = 0.925$. The small degradation under pressure confirms that the trie remains an accurate cache proxy even with a smaller KV cache pool.

**Integrated pipeline.** We evaluate the full integrated pipeline (`VLLMCacheProxy`) against bare vLLM and the simple ordering wrapper.

Within this validation run, the integrated proxy achieves a 29.7% p50 improvement over bare vLLM—comparable to the simple ordering wrapper on the same trace—while adding cache-state synchronization via the feedback loop.

(The 29.7% here is specific to this run's trace; other runs yield 29.6%–32.7% depending on trace seed, per the cross-run note in Section 4.) Per-component profiling shows the system overhead is dominated by prompt construction (0.032 ms) and trie ordering (0.017 ms); inference accounts for 99.9% of wall-clock time.

Under eviction pressure (gpu_mem = 0.78), the feedback loop achieves **99.5% prediction accuracy**—only 1 mismatch out of 200 queries, triggering 1 trie pruning event. The sequential workload produces few evictions even at reduced memory, so the accuracy mainly reflects that the trie is an accurate cache proxy in this regime; the feedback mechanism correctly detects and prunes the rare stale entries that do occur.

## 5 Implementation Details

We release RAGCache++ as a self-contained Python package layered on top of vLLM (≥0.8.0) and FastAPI. The code partitions into two tiers by scope: (i) the **core ordering policy** that all main-benchmark gains rely on—the greedy trie walk in prompt_builder.py (107 LOC) plus the ordering-and-prompt-build portion of serving.py's serve_request loop (∼40 LOC, steps 1–3 of Listing 3)—totals approximately 150 LOC; (ii) the **full integrated system** including the cache manager, PGDSF policy, feedback loop, spatial index, and FastAPI middleware is approximately 2,200 LOC (see Table 12 for the per-file breakdown, which sums to 2,183 LOC; serving.py's ∼250 LOC outside the ordering/prompt-build portion implements the feedback loop, pruning, and HTTP middleware, which are part of tier (ii) but not tier (i)). Source code, benchmarks, and result JSONs are available at https://github.com/tantansir/RAGCachePlusPlus.

### 5.1 Module Organization

*Table 12.* Module organization. Cache subsystem and integration layer are decoupled: the trie + prompt builder can run without GPU for unit tests.

| Module | LOC | Role |
|---|---|---|
| cache/knowledge_tree.py | 232 | Trie + KV metadata |
| cache/cache_manager.py | 615 | Multi-tier accounting |
| cache/pgdsf_policy.py | 153 | PGDSF eviction |
| cache/spatial_index.py | 317 | Geohash spatial filter |
| vllm_integration/prompt_builder.py | 107 | Greedy ordering + prompt |
| vllm_integration/serving.py | 296 | Proxy + feedback + API |
| serving/rag_controller.py | 361 | Pipelining controller |
| config.py | 102 | Config dataclasses |

### 5.2 Key Code Snippets

**Greedy trie-based ordering.** The 25-line realization of Algorithm 1 is the entire "policy" that yields the 20–33% TTFT reduction. It walks the knowledge tree and only follows children whose KV is currently resident (tier != "none"); on the first miss, the remaining documents fall back to retrieval rank order.

*Listing 1.* Greedy ordering (prompt_builder.py). remaining is a set; when multiple cached children exist at one trie level the iteration order is Python-hash-dependent (ties are rare in practice—the trie typically has one matching child per level). The cold-path loop appends every remaining document in retrieval-rank order (matching Algorithm 1).

```
def optimize_doc_order(doc_ids, kt):
    ordered, remaining = [], set(doc_ids)
    node = kt.root
    while remaining:
        found = False
        # Hot path: extend the cached prefix by one
     cached child
        for d in list(remaining):  # set order; tie-
     break is hash-dependent
            child = node.get_child(d)
            if (child is not None
                and child.kv_metadata is not None
                and child.kv_metadata.tier != "none"):
                ordered.append(d); remaining.discard(d
     )
                node = child; found = True
                break   # only break the inner for;
     outer while continues
        if not found:
            # Cold path: append ALL remaining docs in
     retrieval-rank
            # order, then exit the outer while loop.
            for d in doc_ids:
                if d in remaining:
                    ordered.append(d)
            break
    return ordered
```

**TTFT-based mismatch detection.** Without engine instrumentation, the proxy infers vLLM's actual cache state by comparing the trie's predicted reuse fraction with TTFT-implied reuse. A predicted hit ($\hat{r} > 0.3$) that lands near cold-start ($r < 0.15$) is flagged for trie pruning. The cold-TTFT estimate is updated online via EMA over observed misses.

*Listing 2.* Cache-state feedback (serving.py).

```
def check_mismatch(self, predicted_prefix_len, k,
     ttft_ms):
    self.total_observations += 1
    pred_reuse   = predicted_prefix_len / max(k, 1)
    actual_reuse = max(0.0, 1.0 - ttft_ms / max(self.
     cold_ttft, 1.0))
    self.history.append((pred_reuse, actual_reuse))
    if pred_reuse > 0.3 and actual_reuse < 0.15:
        self.mismatches += 1
        return True              # trie thinks hit,
     vLLM evicted
    return False
```

**Integrated serve loop.** The proxy threads ordering, prompt build, vLLM call, feedback, pruning, and admission into a single call with six stages, yielding the per-component overhead numbers reported in Table 3.

*Listing 3.* VLLMCacheProxy.serve_request (abridged).

```
def serve_request(self, query, doc_ids, doc_contents,
     max_tokens=1):
    # 1. Greedy trie walk -> ordered ids + predicted
     prefix length
    ordered, predicted = self._greedy_walk(doc_ids)
    # 2. Build prompt
    prompt, _ = build_rag_prompt(query, ordered,
     doc_contents)
    # 3. vLLM inference (APC enabled at LLM
     construction)
    t0 = time.perf_counter()
    out = self.llm.generate([prompt], sp)
    ttft_ms = (time.perf_counter() - t0) * 1000
    # 4. Feedback loop
    self.feedback.update_cold_estimate(ttft_ms,
     predicted, len(doc_ids))
    miss = self.feedback.check_mismatch(predicted, len
     (doc_ids), ttft_ms)
    # 5. Prune stale paths if vLLM evicted them
    if miss:
        self._prune_path(ordered[:predicted])
    # 6. Admit into multi-tier cache state
    metas = [self.cache_manager.admit(d, 200,
     retrieval_rank=i+1)
             for i, d in enumerate(ordered)]
    self.cache_manager.insert_sequence(ordered, metas)
    self.cache_manager.step()
    return {"ttft_ms": ttft_ms, "predicted_prefix_len"
     : predicted, ...}
```

The proxy is also wrapped as a FastAPI service (`RAGCacheMiddleware`) so an existing RAG application can switch to cache-aware ordering by replacing the `LLM` object with one HTTP endpoint, without touching its serving stack.

### 5.3 Engineering Details and Trade-offs

- **vLLM API surface.** We pin to vLLM $\geq$ 0.8.0 for stable APC behavior; the proxy invokes only the public `LLM.generate` interface with `enable_prefix_caching=True` and standard `SamplingParams`. No internal vLLM symbols are imported, which makes the integration robust to engine-version drift.
- **Trie memory footprint.** Nodes use `__slots__` to bound per-node memory at $\sim$120 B. In our 200-query benchmark the trie holds $<$1 MB, well below any meaningful host budget.
- **Reproducible eviction.** `CacheManager.step()` advances a logical timestamp consumed by the PGDSF priority, decoupling eviction order from wall-clock noise. All benchmark scripts accept a `seed` argument; the multi-seed statistics in Section 4 are produced by re-running with seeds $\{0, 1, 2\}$ and aggregating with a $t$-distribution 95% CI.
- **Fallback path.** When the trie is empty (cold start) or no document matches a cached child, the algorithm degenerates to retrieval-rank order. *Algorithmically*, this makes the proxy no worse than bare APC in the fallback case; empirical TTFT can still differ by a few milliseconds of measurement noise across independent runs.

## 6 Discussion

**Prompt-layer cache scheduling.** RAGCache++ introduces *prompt-layer cache scheduling* for APC-based RAG serving, analogous to how disk I/O schedulers reorder requests to exploit spatial locality without modifying the disk firmware. The ordering policy does not modify the cache mechanism itself (APC) but determines which cache lines are populated and in what order, directly controlling the prefill–reuse trade-off. Our profiling confirms this is a genuine systems effect: optimized ordering saves an estimated 66% of prefill computation at p50 (TTFT-based proxy, Table 6), reduces inference latency by 29% (Table 3), with no material throughput regression ($<$0.4%) and zero memory overhead. The key architectural insight is that a *thin scheduling layer* between retrieval and serving can recover substantial cache locality that the retrieval stage destroys—without requiring engine-level changes. This makes the policy composable with any APC-enabled system and deployable as a drop-in middleware component. We emphasize that the contribution is *cache-aware document ordering as a scheduling abstraction*. The trie-based greedy algorithm achieves 97.5% of oracle TTFT reduction with $O(k^2)$ worst-case cost ($O(k)$ typical) (prefix-length optimality ratio 0.997), and its online adaptivity enables it to track shifting workloads—making it the practical default for deployment. We acknowledge that the core scheduling idea is shared with concurrent work (ContextPilot (Wang et al., 2025)); our contribution is the *deployment-ready realization*: a self-contained, engine-agnostic implementation with empirically measured near-optimality (prefix-length ratio 0.997, 97.5% of oracle TTFT reduction), a closed-loop feedback mechanism, and comprehensive cross-workload/cross-architecture evaluation demonstrating that this simple approach is sufficient for practical RAG serving.

**Relation to RAGCache.** Our knowledge tree is inspired by RAGCache's (Jin et al., 2025) document-indexed trie structure, but serves a different purpose. RAGCache uses the tree for custom KV cache management within a modified serving engine; we use it solely for ordering decisions, relying on the engine's built-in APC for cache management. Our trie-based ordering achieves 97.5% of oracle's TTFT reduction with $O(k^2)$ worst-case cost ($O(k)$ typical), confirming near-optimal scheduling quality. Our approach is complementary to RAGCache: it achieves comparable scheduling benefits without requiring engine modifications, and could be combined with RAGCache's cache management for further gains.

**Relation to CacheBlend.** CacheBlend (Yao et al., 2025) addresses a harder problem: reusing KV blocks regardless of position via selective recomputation. Our approach avoids this complexity entirely by converting the problem to pure prefix matching through ordering. The two approaches are

complementary: CacheBlend could handle the non-prefix portion of our reordered sequences for further gains.

**Limitations.**

- **Controlled workloads**: Our main benchmark uses synthetic documents with controlled overlap. We supplement with a TF-IDF-retrieved diverse corpus (Section 4.6), a geospatial workload (Section 4.5), and NQ-Open traces (Section 4.6), but production multi-tenant RAG traces remain future work.
- **Concurrency scope**: Batched evaluation (Table 7) shows ordering benefits at concurrency 1–8, and cache pressure sweeps confirm robustness across memory levels. However, we do not evaluate under sustained multi-tenant load with realistic arrival distributions (e.g., Poisson arrivals, heterogeneous query patterns), where trie staleness and scheduling interactions may differ.
- **Throughput is decode-dominated**: Ordering improves prefill, not decode. Under workloads with long output sequences (e.g., summarization), the TTFT benefit shrinks as a fraction of total latency.
- **Model family**: We evaluate Qwen2.5 at three scales (0.5B, 1.5B, 7B) and Microsoft Phi-3 (3.8B), observing consistent 18–33% benefits across architectures. Validation on further model families (e.g., Llama, Mistral) remains future work.

**Future work.** Promising extensions include: (1) combining ordering with CacheBlend-style selective recomputation for non-prefix documents, (2) extending the TTFT-based feedback loop to query vLLM's block table directly for more precise cache-state tracking under sustained high-concurrency load, (3) real-corpus evaluation on production RAG workloads with natural temporal locality, and (4) multi-engine validation (SGLang RadixAttention).

## 7 Conclusion

We presented RAGCache++, a prompt-layer scheduling policy that improves KV cache prefix reuse in RAG serving by reordering documents to extend cached prefixes. Under bursty document overlap, RAGCache++ reduces median TTFT by 20–33% over retrieval-order APC across two GPU configurations (statistically significant: multi-seed p50 improvement $16.8 \pm 1.6$ ms, 95% CI excludes zero), saving an estimated 66% of prefill computation at p50 (TTFT-based proxy) while adding <0.13% overhead, no material throughput regression (<0.4%), and zero GPU memory cost. End-to-end pipeline measurements confirm that benefits persist when retrieval latency is included (16–26% at batch size 1 across $T_{\text{ret}} = 10$–$50$ ms; 2.7–9.5% at higher batch sizes). The knowledge tree's predicted reuse correlates with observed reuse at $r = 0.97$, validating the approximate cache index design. The greedy ordering achieves 97.5% of oracle's TTFT reduction (prefix-length optimality ratio 0.997), benefits persist under concurrent load (7–10% at batch sizes 1–8) and are robust to cache pressure (29–31% across GPU memory levels). On a geospatial workload with naturally lower overlap (Jaccard $= 0.17$), ordering still provides measurable gains (4.5% TTFT reduction). The approach requires no serving-engine modifications ($\sim$150 LOC for the core ordering policy; $\sim$2,200 LOC for the full integrated system). Our results demonstrate that prefix *depth*—not just hit count—is the key driver of latency reduction in prefix-caching systems.

## 8 Team Member Contributions

The project was led by **Kaizhen Tan**, with **Rong Gu** and **Mingyuan Li** each owning specific subsystems and experiment tracks.

**Kaizhen Tan.** Owned the architecture and integration layer (`serving.py`, `prompt_builder.py`, benchmark drivers), wired the stand-alone cache modules into vLLM through the 6-stage `serve_request` loop, and ran the main TTFT, systems-analysis, end-to-end, feedback-loop, and cross-architecture experiments on both the RTX 4060 Ti and AutoDL RTX 4090. Primary author of the Abstract, Introduction, Methodology, Experiments, and Implementation Details sections; produced all figures and set up the GitHub repository.

**Rong Gu.** Owned the cache subsystem (`knowledge_tree.py`, `cache_manager.py`, `pgdsf_policy.py`, `spatial_index.py`). Designed and ran the system-validation and baseline-matrix experiments (Section 4.8, Table 8). Co-authored the Background and Methodology subsections on APC mechanics, the knowledge tree, and the cache-state feedback loop.

**Mingyuan Li.** Owned workload generation, retrieval, and the high-level controller (`workload_generator.py`, `retrieval/`, `rag_controller.py`). Ran the answer-quality and workload-diversity experiments (multi-hop, NQ-Open, mixed-topic, real corpus, geospatial). Lead author of Related Work, Discussion, and Conclusion; curated `references.bib` and wrote the README.

## References

Agarwal, A., Mitra, S., Jain, S., and Gupta, P. Cache-Craft: Managing chunk-caches for efficient retrieval-augmented generation, 2025.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., and Wang, H. Retrieval-augmented generation

for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.

Jin, C., Zhang, Z., Jiang, X., Liu, F., Liu, X., Liu, X., and Jin, X. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 44(1):1–27, 2025. doi: 10.1145/3768628.

Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., Toutanova, K., Jones, L., Kelcey, M., Chang, M.-W., Dai, A. M., Uszkoreit, J., Le, Q., and Petrov, S. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019. doi: 10.1162/tacl_a_00276.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention, 2023.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Kundu, S., Du, K., Lu, S., and Jiang, J. CacheGen: KV cache compression and streaming for fast large language model serving. *ACM SIGCOMM*, 2024.

Lu, F., Yao, Y., Tong, X., Lin, Y., Zhang, R., Chen, J., and Xie, S. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text, 2024.

Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.

Tang, Y., Wang, Z., Qu, A., Yan, Y., Wu, Z., Zhuang, D., Kai, J., Hou, K., Guo, X., Zhao, J., Zhao, Z., and Ma, W. ItINERA: Integrating spatial optimization with large language models for open-domain urban itinerary planning. In *EMNLP Industry Track*, 2024.

vLLM Project. Automatic Prefix Caching. https://docs.vllm.ai/en/latest/design/prefix_caching/, 2025. Accessed: 2026-03-31.

Wang, Y., Sheng, Y., Zheng, L., and Stoica, I. ContextPilot: Fast long-context inference via context reuse, 2025.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, Rotterdam, Netherlands, 2025. ACM. doi: 10.1145/3689031.3696098.

Yu, D., Bao, R., Ning, R., Peng, J., Mai, G., and Zhao, L. Spatial-RAG: Spatial retrieval augmented generation for real-world geospatial reasoning questions, 2025.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kober, C., Shi, C., Zhuang, S., Gonzalez, J. E., Barrett, C., and Stoica, I. SGLang: Efficient execution of structured language model programs, 2024.

## A Experimental Configuration Details

*Table 13.* Hardware and software configuration.

| | **Config A** | **Config B** |
|---|---|---|
| GPU | 4060 Ti 8 GB | 4090 24 GB |
| Model | Qwen2.5-1.5B | Qwen2.5-7B |
| vLLM | 0.8.5.post1 | 0.17.1 |
| `max_model_len` | 2048 | 4096 |
| `gpu_mem_util` | 0.85 | 0.90 |
| Corpus | 100 docs | 500 docs |
| Queries | 100 | 200 |
| Top-$k$ | 5 | 5 |
| Use | TTFT, overlap | All (§4–4.8) |

## B Overlap Sweep Raw Data

*Table 14.* Raw data for overlap sensitivity (RTX 4060 Ti, Qwen2.5-1.5B-Instruct).

| **Overlap Frac.** | **Jaccard** | **Retrieval p50 (ms)** | **Optimized p50 (ms)** | **Improv.** |
|---|---|---|---|---|
| 0.0 | 0.386 | 126.8 | 111.7 | 11.9% |
| 0.2 | 0.386 | 128.9 | 114.2 | 11.5% |
| 0.4 | 0.417 | 111.1 | 77.8 | 30.0% |
| 0.6 | 0.526 | 75.5 | 60.5 | 19.9% |
| 0.8 | 0.661 | 50.4 | 50.8 | −0.8% |

## C Systems Benchmark Details

The systems analysis in Section 4.5 (throughput, profiling, pipelining, memory) uses Config B (RTX 4090) with `enforce_eager=True` (CUDA graphs disabled for measurement stability).